

Fine-Tuning Revisited: Efficient LLM Adaptation with LoRA

Applied Deep Learning — Chapter 13

Yin Yang

Foundation Books Project

Roadmap

Running Example and Goals

Why Full Fine-Tuning Is Expensive

LoRA: Low-Rank Adaptation

Quantization and QLoRA

Supervised Fine-Tuning Data

Unsloth Workflow

Adapter Artifacts and Deployment

Practical Recipe

Failure Modes and Debugging

Summary

Running Example and Goals

The Study Assistant

Earlier chapters covered prompt-only inference and retrieval. This chapter handles behavior *retrieval cannot fix*.

- Switches answer format mid-conversation.
- Over-explains beginner questions.
- Forgets the warning about keeping the test set untouched.

Desired behavior: textbook terminology, discourage test-set tuning, concise answers, practical advice for beginners.

Question: can a small trainable component reduce these failures on held-out prompts, enough to justify added complexity?

Adapters and Efficient Adaptation

Adapter

A small trainable component added to (or associated with) a pretrained network so that most original weights remain frozen.

- LoRA: low-rank trainable updates for selected weight matrices.
- QLoRA: same adapter idea, but base model is loaded quantized.
- Goal: cheaper, smaller, easier-to-store adaptations.

Efficient is not automatic. Fewer trainable parameters = cheaper, not automatically better. Validation still decides.

First Intervention?



- Format unstable but facts present $\Rightarrow$ try LoRA.
- Facts missing $\Rightarrow$ try retrieval first.
- Memory tight $\Rightarrow$ QLoRA.
- Full fine-tuning is rarely the right first move at small-project scale.

Why Full Fine-Tuning Is Expensive

The Memory Budget

Eight-billion-parameter model in 16-bit precision:

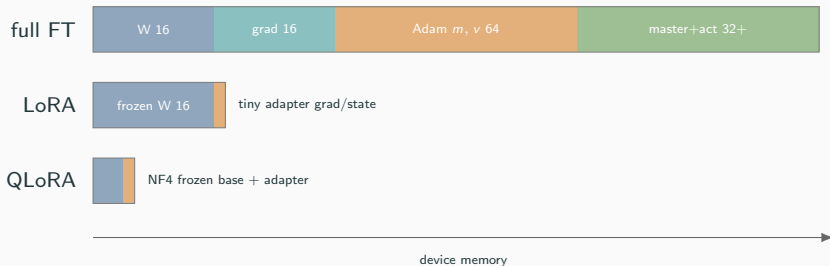
$$8 \times 10^9 \times 2 \text{ bytes} = 16 \text{ GB.}$$

Inference is one line item. Full mixed-precision Adam training adds:

Component	bytes/param	8B params
16-bit weights	2	16 GB
16-bit gradients	2	16 GB
Adam first moment	4	32 GB
Adam second moment	4	32 GB
32-bit master weights (opt.)	4	32 GB

$\approx$ 96–128 GB before activations and context.

Memory Is a Budget, Not One Number



LoRA changes only the trainable-parameter portion. Frozen base still loads for forward pass; gradients and optimizer state shrink to the adapter.

Trainable Parameter Count

Definition

The number of parameters whose values the optimizer is allowed to update. Frozen parameters still participate in the forward pass.

Adapter accounting for a 32-layer model adapting four 4096×4096 projections per layer at rank $r = 8$:

$$32 \times 4 \times (8 \times 4096 + 4096 \times 8) = 8,388,608.$$

About 8.4M trainable parameters instead of 8B.

Risk of full fine-tuning isn't only hardware: with a small dataset the optimizer can overwrite useful pretrained behavior.

LoRA: Low-Rank Adaptation

The LoRA Update

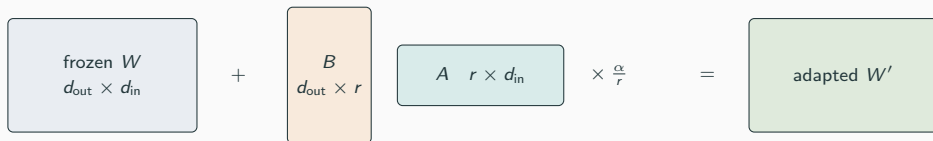
For pretrained $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$:

$$W' = W + \frac{\alpha}{r} BA,$$

with $A \in \mathbb{R}^{r \times d_{\text{in}}}$, $B \in \mathbb{R}^{d_{\text{out}} \times r}$, rank r , scale α .

- W frozen; only A and B trainable.
- Forward pass: $W'x = Wx + \frac{\alpha}{r} B(Ax)$.
- Optionally merged into W at deployment.

LoRA Diagram



- $r \ll d_{\text{in}}, d_{\text{out}}$ keeps BA a low-rank correction.
- Only A and B receive gradients.
- Stored as adapter; can also merge later.

Parameter Count: Concrete Example

Adapted matrix with $d_{\text{in}} = d_{\text{out}} = 4096$. Full fine-tune:

$$4096 \times 4096 = 16,777,216 \text{ parameters.}$$

LoRA at $r = 8$:

$$8 \times 4096 + 4096 \times 8 = 65,536 \text{ parameters.}$$

About 0.39% as many — per matrix. Real models adapt several projections per layer; the total is larger but still tiny compared with full fine-tuning.

Target Modules Matter

Decoder-only LLM attention projections:

Common first targets q_proj, k_proj, v_proj, o_proj.

Feed-forward gate_proj, up_proj, down_proj — more capacity, more cost.

Report the adapter. Rank alone is not enough — target modules and alpha are part of the method.

More target modules = more trainable parameters and more overfitting risk.

Knobs and Conservative Defaults

Knob	What it changes	Conservative first use
Rank r	capacity per adapted matrix	start at 8 or 16; ablate up only if underfitting
Alpha α	scale of update via α/r	set to rank or a small multiple; do not vary with rank
Dropout	regularization on adapter branch	zero or small; raise if memorized/repetitive
Target modules	which matrices get adapters	start with attention; add MLPs only if needed

Spend the first run on measurement, not cleverness.

Quantization and QLoRA

Definition

Quantization maps high-precision values to a smaller set of representable values or codes. Stored compactly; computation uses approximate values.

Simple uniform quantizer over a block:

$$q(w) = \text{round}\left(\frac{w}{s}\right), \quad \hat{w} = s q(w), \quad e_q = w - \hat{w}.$$

$q(w)$ is the stored code; $\hat{w}$ is what computation uses; e_q is the quantization error.

Real 4-bit loaders use blockwise scales, zero-points or codebooks, and library-specific kernels.

Memory: 16-bit vs 4-bit

One billion parameters:

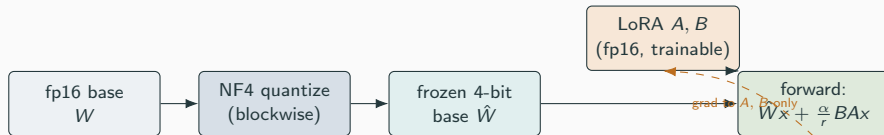
- 16-bit: ≈ 2 GB raw weight storage.
- 4-bit: ≈ 0.5 GB before metadata and overhead.

Real runtime memory depends on:

- scales, codebooks, temporary dequantized values
- activations, attention buffers
- library implementation details

Direction is clear: **fewer bits** can make a larger frozen base feasible.

QLoRA in One Picture



Not “training in 4-bit.” Base loaded compressed; adapter trains in an ordinary differentiable workflow.

Find nearest codebook value v_m :

$$c = \arg \min_{m \in \{1, \dots, 16\}} |w - v_m|, \quad \hat{w} = v_c.$$

$v_1, \dots, v_{16}$: 16 representable NF4 values after block scaling. **Double quantization:** compress the metadata (scales) too. Per-param savings are small but at billions of parameters they matter.

Paged optimizer: system-level mitigation for memory spikes — not a modeling guarantee.

What to Report for QLoRA

Ingredient	Memory role	What to report
4-bit frozen base (NF4)	raw storage savings	checkpoint, quantization type, compute dtype, loader
Double quantization	metadata overhead	whether enabled, package version
LoRA adapters	focus optimizer state	rank, alpha, dropout, target modules, trainable count
Paged optimizer	spike handling	optimizer name, batch, accumulation, seq length, peak mem

“QLoRA rank 16” is too thin for a lab report.

Supervised Fine-Tuning Data

SFT and Chat Templates

Definition

Supervised fine-tuning (SFT) adapts a pretrained model on examples that contain desired outputs. For chat models, examples are formatted as system/user/assistant messages before tokenization.

Template changes are data changes. Compare runs only after the rendered chat format is held fixed.

Inspect each example in three forms:

1. human-readable messages
2. rendered chat text from `apply_chat_template`
3. tokenized: input IDs, attention mask, labels

Loss Masking

Definition

Loss masking controls which tokens contribute to the SFT loss. Ignored tokens are assigned label -100.

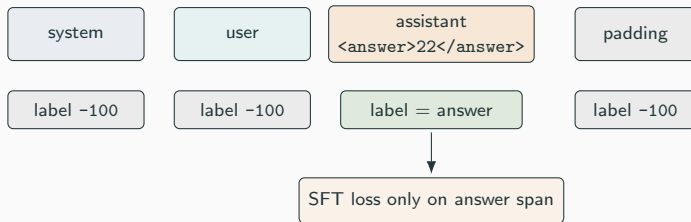
Masked SFT loss with $m_t \in \{0, 1\}$:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{\sum_t m_t} \sum_t m_t \log p_{\theta}(y_t \mid y_{<t}, x).$$

Wrong mask = different objective:

- Mask includes prompt $\Rightarrow$ teaches model to reproduce questions.
- Mask removes answer $\Rightarrow$ trains nothing useful.
- Padding not masked $\Rightarrow$ learns batching artifacts.

SFT Token Spans



End-of-sequence tokens tell the model where the answer stops; missing EOS lets a trained adapter run on into the next turn.

Padding shares length within a batch; must be ignored by labels and attention.

Truncation dangerous when the end of the answer is clipped.

Packing concatenates short examples; needs clear boundaries and stop tokens.

Duplicate / near-duplicate examples = false confidence:

- training: model memorizes phrase pattern
- validation: rewrites of training prompts $\Rightarrow$ measures memorization

Unslow Workflow

Unsloth is tooling, not a new algorithm.

- Wraps and accelerates Hugging Face + PEFT workflow.
- Loads supported models, optionally with 4-bit quantized base.
- Provides `FastLanguageModel` entry point.

Tooling is not evidence. Validation data and controlled comparisons still decide whether the adapter helped.

The Library Stack

Package	Responsibility	Record
Transformers	model + tokenizer + TrainingArguments	version, checkpoint, template, precision
Datasets	JSONL → table for SFT	split, counts, fields, dedup checks
TRL	SFTTrainer loop	batch, accumulation, LR, steps, packing, masking
PEFT/LoRA	adapter config and trainable params	rank, alpha, dropout, targets, trainable count, merge
Unsloth	loaders + adapter attach	Unsloth ver., load_in_4bit, ckpt setting

Key API Calls (and What They Decide)

`FastLanguageModel.from_pretrained` checkpoint, max seq length, precision,
 `load_in_4bit`.

`tokenizer.apply_chat_template` formats each example as the chat text expected by the
 model.

`FastLanguageModel.get_peft_model` sets rank, alpha, dropout, target modules, seed,
 gradient checkpointing.

`SFTTrainer + TrainingArguments` batch, gradient accumulation, LR, optimizer, steps,
 warmup, seed.

`model.save_pretrained` writes adapter artifacts and tokenizer to the run directory.

Adapter Artifacts and Deployment

What an Adapter Run Saves

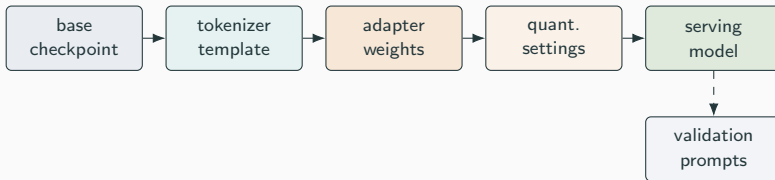
Adapter artifact

The saved output of a parameter-efficient run. Adapter weights, adapter configuration, and metadata to reconnect to the correct base. *Not, by itself, a full language model.*

Reload requires:

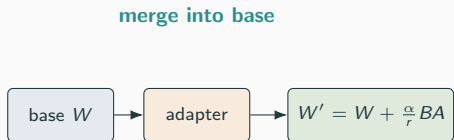
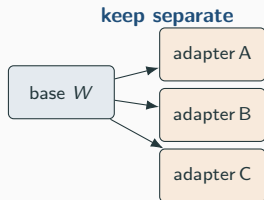
- base checkpoint name/revision and license constraints
- tokenizer revision and chat template
- adapter weights and configuration
- quantization mode and loader for QLoRA

Adapter Loading Context



Wrong base, tokenizer, or template can change behavior even when loading succeeds. Reload checks must compare *behavior*, not just file existence.

Merging vs Keeping Separate



Keep separate: one base + many adapters; swap cheaply; smaller artifact.

Merge: one set of weights; simpler serving; larger artifact; loses base/task separation.

Validate the merged model as a new serving object.

The Deployment Sanity Check

Loading an adapter successfully is not evidence it is the right adapter. After load, run:

1. a few fixed validation prompts
2. compare with the recorded adapter run
3. fail loudly if base, tokenizer, adapter path, or decoding mismatch

Catches:

- serving the base model without the adapter
- serving the wrong adapter
- right adapter on incompatible base
- template changed between training and inference

Practical Recipe

Start From a Good Baseline

1. Prompt-only baseline: same checkpoint, tokenizer, validation prompts, template, decoding, rubric.
2. If facts are missing $\Rightarrow$ retrieval before training.
3. If style/format is unstable $\Rightarrow$ test one LoRA adapter.
4. Ablate one factor at a time.

The first field to check in an adapter report is not rank — it is whether baseline and adapter used the same rendered prompt and decoding rule.

Bounded Reference Run

Base Qwen/Qwen2.5-0.5B-Instruct.

Run	Setup	Trainable	Time / memory
Baseline	no adaptation, 5 prompts	0	0.079 s first tok; 69.0 tok/s
Rank-8	LoRA on q, k, v, o ; 384-tok cap	1,081,344	2.35 s; 1.24 GB reserved
Rank-16	same except r, α	2,162,688	2.37 s; 1.25 GB reserved

Rank-16 loss fell farther in this smoke run — but this is a feasibility check, not a deployment recommendation.

What a Useful Conclusion Names

Beyond “it improved”:

- validation score on held-out prompts
- trained-parameter percentage
- time and memory
- failure pattern that remained

An adapter that fails to beat the baseline is evidence for:

- better data
- retrieval
- a stronger base model
- a clearer prompt

Not for “more training.”

Failure Modes and Debugging

Symptom → Cause

Symptom	Likely cause
Train loss down, val answers worse	memorization, duplicate examples, too many steps, too much capacity
Model copies prompt or boilerplate	wrong loss mask, unwanted boilerplate in examples
Answers run on or merge examples	missing EOS, bad packing, wrong inference template
Adapter loads but behaves like base	wrong path, inactive adapter, incompatible base
Slower / more memory than expected	long context, more targets, no checkpointing, wrong precision

The Useful Debugging Pass

Before increasing rank or training longer:

1. inspect one formatted training example
2. decode one tokenized batch with labels
3. run an overfit-one-example check
4. compare prompt-only on the same held-out prompts
5. reload the saved adapter

Then change *one* thing: rank, target modules, examples, steps, learning rate, max sequence length, or quantization mode.

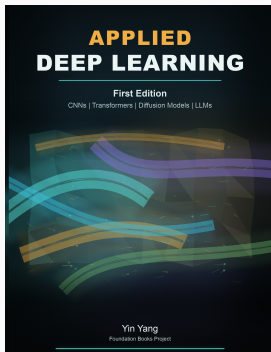
If the task mainly needs fresh facts: retrieval or a better prompt contract is the more direct fix.

Summary

Summary

- **Full fine-tuning** is flexible but expensive: optimizer state, activations, context length, overfitting risk.
- **LoRA** freezes W and trains $\frac{\alpha}{r}BA$ with $rd_{\text{in}} + d_{\text{out}}r \ll d_{\text{out}}d_{\text{in}}$.
- Report rank, alpha, target modules, dropout, trainable count, time, memory.
- **QLoRA** keeps the adapter idea on a quantized frozen base. Memory win depends on precision, dtype, optimizer, sequence length, and software stack.
- **SFT** cares about chat templates, assistant-token loss masking, EOS, padding, truncation, packing, duplicates.
- **Evidence still comes from** reserved validation prompts, a prompt-only baseline, fixed decoding, tokenized-batch inspection, one-factor ablations, and adapter reload checks.

Reload the adapter, run the same prompt and retrieved context, and prove that the reported behavior comes from the intended base-plus-adapter setup.



Applied Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	foundation-books.github.io/applied-deep-learning
Code	github.com/foundation-books/applied-deep-learning-code
Print	amazon.com/dp/B0GZF7QRSH
Kindle	amazon.com/dp/B0GZF9221X
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.